Hoja de Ruta y Backlog de Remediación Frontend

XelerGo Frontend

Propósito del documento

Definir una hoja de ruta estrictamente del lado frontend para corregir los hallazgos prioritarios de la auditoría y transformar el estado actual en una base técnicamente más segura, mantenible y confiable.

Tipo de documento	Roadmap y backlog técnico / funcional del frontend
Fuentes base	AUDITORIA_QA_FRONTEND_VUE.pdf y RESUMEN_EJECUTIVO_AUDITORIAS_XELERGO.pdf
Alcance	Solo frontend. No duplica la hoja de ruta backend ya planteada.
Fecha	23/03/2026
Stack evaluado	Vue 3 + TypeScript + Pinia + Vuetify
Autor	Creado por Jose Manuel Guerrero Duarte, Ing en informática

Documento aislado solo front

1. Objetivo, alcance y criterio de uso

Este documento traduce los hallazgos del frente cliente en una secuencia de trabajo ejecutable y priorizada.

La hoja de ruta parte de un diagnóstico ya establecido: el frontend es una base operable para evolución, pero no está listo para aprobación de producción por seguridad del cliente, navegación frágil, validaciones debilitadas, flujos cripto incompletos, manejo irregular de errores y ausencia de un soporte mínimo de no regresión.

- El alcance es exclusivamente frontend. No replica tareas propias del backend ni reabre la auditoría del servidor.
- Cuando una actividad frontend depende de una capacidad ya existente o pendiente en backend, aquí solo se documenta la parte cliente: consumo de contrato, representación honesta del estado, validación local y manejo seguro del fallo.
- La prioridad no es ampliar alcance visual. La prioridad es bajar riesgo de producción, corregir puntos engañosos de la UX y reducir la probabilidad de regresión al tocar funcionalidades sensibles.

Estado de partida

Base útil para refactor y evolución controlada, pero no aprobable hoy para producción.

Bloqueos principales

Seguridad del token, guards, validaciones críticas, flujo cripto real en UI, manejo de errores y cobertura mínima.

Enfoque recomendado

Contener riesgo primero, cerrar flujos críticos después y recién entonces consolidar arquitectura y calidad.

Horizonte sugerido

5 a 6 sprints para un equipo frontend pequeño con dedicación real. La secuencia es más importante que la fecha exacta.

Qué sí persigue esta hoja

Dejar una ruta concreta para que el frontend pase de una V1 visualmente avanzada, pero frágil, a una base más segura y mantenible. El foco está en contratos de UI, navegación, validaciones, estados y calidad técnica.

Qué no debe interpretarse

No es una promesa de release date ni una certificación de que todos los riesgos desaparecerán por sí solos. Tampoco reemplaza las tareas backend, regulatorias o operativas que ya tienen su propia hoja de ruta.

2. Criterios de priorización

La secuencia propuesta sigue impacto de negocio, exposición al riesgo y capacidad de reducir regresiones.

- Primero se corrigen los puntos que comprometen seguridad, autorización y confianza básica del usuario.

- Después se cierran los flujos que hoy aparentan más de lo que realmente soportan, en especial los vinculados a compra, seguimiento y wallet XCOP.
- Más adelante se ataca el acoplamiento estructural para que el sistema pueda evolucionar sin depender de vistas gigantes, helpers transversales y side effects dispersos.
- La última fase consolida pruebas, lint y criterios de salida para evitar que el saneamiento se pierda en iteraciones siguientes.

Decisión de producto que debe resolverse en el Sprint 0

El equipo debe definir explícitamente si el frontend va a exponer de verdad el flujo cripto externo por proveedor en esta etapa. Si la respuesta es sí, las tareas de cierre funcional de wallet XCOP, timeline y estados reales pasan a ser bloqueantes de release. Si la respuesta es no, la UI debe reducir alcance, etiquetar correctamente lo soportado y retirar señales que hoy inducen a pensar que el ciclo externo ya está resuelto.

3. Hoja de ruta por fases

Secuencia sugerida para ejecutar el backlog sin mezclar saneamiento crítico con refactors no prioritarios.

Fase	Horizonte orientativo	Resultado esperado	Incluye
Fase 0	**Sprint 0 a 1**	Contención inmediata de riesgo y corrección de la base de navegación y sesión.	Política HTTP segura, guards determinísticos, persistencia de sesión revisada y manejo de errores críticos sin comportamientos peligrosos.
Fase 1	**Sprint 1 a 3**	Verdad funcional en los flujos sensibles del usuario y UX alineada al alcance real del producto.	Validaciones activas, wallet XCOP honesta respecto a su alcance, timeline conectado a estados reales y feedback consistente en compra, venta y transacciones.
Fase 2	**Sprint 3 a 5**	Reducción del acoplamiento estructural y mejor mantenibilidad de la base frontend.	Extracción de servicios y composables, división de piezas sobredimensionadas y eliminación de side effects en componentes reutilizables.
Fase 3	**Sprint 5 a 6**	Capa mínima de calidad y no regresión para sostener la evolución posterior.	Línea base de lint, pruebas críticas, smoke tests y checklist técnico previo a reevaluar aprobación de producción.

Los refactors visuales no críticos, la optimización fina de performance y la limpieza de pantallas públicas de bajo riesgo no deben competir por capacidad con estas cuatro fases. Pueden diferirse siempre que no vuelvan a contaminar las piezas compartidas ni las rutas sensibles.

4. Backlog priorizado

Las actividades están agrupadas por frente. Cada una deja claro qué debe cambiar del lado frontend para poder considerarla cerrada.

Frente A. Seguridad, sesión y navegación

ID	Pri	Actividad	Criterio de cierre / entregable frontend
FE-01	**P0**	Separar el cliente HTTP interno del consumo de servicios externos y bloquear el envío de Authorization a URLs absolutas de terceros.	El frontend distingue llamadas internas y externas; ninguna request a dominios públicos reutiliza el bearer token; queda verificado al menos en precios, ticker y broadcasting.
FE-02	**P0**	Reordenar la estrategia de sesión para reducir dependencia visible de localStorage en token, usuario y banderas de 2FA.	Persistencia más controlada, limpieza consistente al cerrar sesión y reglas claras sobre qué se conserva, qué se recalcula y qué nunca debe quedar como bandera sensible en storage.
FE-03	**P0**	Reescribir beforeEach y middleware de rol, permiso y verificación con retornos explícitos y un único desenlace por navegación.	Las rutas resuelven acceso, redirección o rechazo sin dobles next, sin ramas ambiguas y con comportamiento consistente para invitado, autenticado, 404 y rutas restringidas.
FE-04	**P0**	Normalizar manejo de errores en helper y vistas críticas para evitar console-only catch, accesos inseguros a error.response y recargas completas como cierre de flujo.	Los errores de red o API muestran feedback coherente, no exigen recargar la página para recomponer estado y dejan trazabilidad mínima para soporte y QA.

Frente B. Flujos críticos y verdad funcional

ID	Pri	Actividad	Criterio de cierre / entregable frontend
FE-05	**P0**	Reactivar y endurecer la validación real de dirección en compra, contemplando moneda, red y método de entrega soportado.	La dirección no pasa por return true; el formulario bloquea datos inválidos, muestra mensajes útiles y no confirma la orden mientras la validación local no sea consistente.
FE-06	**P1**	Alinear la pantalla de recepción wallet XCOP con el alcance real del producto.	La UI deja de insinuar un depósito externo resuelto si solo soporta recepción interna. El texto, la interacción y el dato generado son honestos respecto a lo que realmente hace el frontend.
FE-07	**P1**	Alinear el retiro wallet XCOP con el alcance real soportado desde frontend.	El usuario distingue con claridad si el retiro disponible es hacia cuentas del cliente o hacia wallet externa. Desaparece la ambigüedad actual y el formulario pide exactamente lo que el flujo soporta.

ID	Pri	Actividad	Criterio de cierre / entregable frontend
FE-08	P1	Sustituir timeline y seguimiento estático por estados de transacción realmente reflejados en pantalla.	La línea de tiempo deja de depender de textos o tiempos fijos; consume cambios reales de orden, limpia polling y suscripciones, y representa avance, espera, error y cierre de forma creíble.
FE-09	P1	Estandarizar loaders, estados vacíos, submitting, retry y mensajes de fallo en compra, venta, wallet XCOP y transacciones.	Los flujos críticos comparten una política visual consistente ante carga, validación, error recuperable y fallo terminal; se reducen pantallas frágiles y decisiones implícitas del usuario.

Frente C. Arquitectura y mantenibilidad

ID	Pri	Actividad	Criterio de cierre / entregable frontend
FE-10	P1	Dividir la integración actual en servicios o clientes por dominio funcional: autenticación, órdenes, wallet XCOP, catálogos y utilitarios.	Las vistas y stores dejan de depender de strings de endpoint dispersos y del helper transversal como punto único de acoplamiento; cada dominio expone una interfaz más clara.
FE-11	P2	Extraer composables y reducir lógica embebida en vistas y componentes de alto riesgo.	Timeline, compra, wallet XCOP y piezas similares trasladan lógica reutilizable fuera del template y del setup principal, con responsabilidades más pequeñas y testeables.
FE-12	P2	Eliminar mutación de props y side effects en componentes reutilizables.	Selects, tablas y wrappers globales trabajan con estructuras derivadas o clonadas, sin modificar props recibidas ni introducir efectos laterales ocultos.
FE-13	P2	Reducir deuda visible en piezas transversales sobredimensionadas y supuestos de UI estáticos.	Los archivos más sensibles quedan segmentados, con menos responsabilidad múltiple, menos intervalos huérfanos, menos textos hardcodeados engañosos y menor costo de cambio.

Frente D. Calidad, pruebas y criterio de salida

ID	Pri	Actividad	Criterio de cierre / entregable frontend
FE-14	P1	Definir una línea base de lint que deje de permitir crecimiento libre de la deuda actual.	Queda establecida una estrategia de saneamiento incremental: reglas mínimas obligatorias, exclusiones justificadas y bloqueo de nueva deuda en archivos tocados.
FE-15	P1	Crear pruebas automáticas para guards, política HTTP, validadores críticos y sesión.	Existen pruebas unitarias o equivalentes para navegación, autorización básica, validación de dirección y comportamiento del cliente HTTP frente a dominios externos.

ID	Pri	Actividad	Criterio de cierre / entregable frontend
FE-16	P2	Cubrir con pruebas componentes y stores sensibles de compra, venta, wallet XCOP y seguimiento.	Los estados relevantes de formularios, errores, transiciones y rendering crítico cuentan con respaldo automatizado suficiente para detectar regresiones frecuentes.
FE-17	P2	Montar smoke tests o e2e mínimos para login, 2FA, creación de orden, seguimiento y flujo XCOP dentro del alcance realmente soportado.	QA y desarrollo disponen de una verificación automática básica de punta a punta para no depender solo de pruebas manuales exploratorias.
FE-18	P1	Implantar criterio técnico previo a reevaluar producción: build, typecheck, lint acordado, suite mínima y checklist manual.	No se reabre discusión de aprobación hasta que el frontend pueda demostrar una salida repetible, medible y defendible frente a seguridad, flujo crítico y no regresión.

5. Condiciones mínimas de cierre por fase

La fase se considera terminada cuando el frente cliente demuestra comportamiento observable, no solo código movido.

- Fase 0 cerrada: el token ya no viaja a terceros, la navegación resuelve de manera determinística y los errores críticos no dependen de recargar pantalla ni de console.error como único desenlace.
- Fase 1 cerrada: compra vuelve a validar direcciones de forma real, wallet XCOP representa con honestidad lo que soporta, y el seguimiento muestra estados transaccionales confiables en lugar de un timeline decorativo.
- Fase 2 cerrada: los dominios sensibles ya no dependen de vistas gigantes ni de un helper único para toda la integración, y los componentes reutilizables dejan de mutar datos recibidos.
- Fase 3 cerrada: existe un mínimo razonable de pruebas y de validación estática para impedir que el saneamiento se revierta en la siguiente iteración.

Qué puede diferirse

Optimización fina de bundles, refactors visuales de páginas públicas, mejoras cosméticas del dashboard y limpieza profunda de áreas no críticas, siempre que no interfieran con piezas compartidas.

Qué no debería diferirse

Seguridad del token, guards, validaciones críticas, verdad funcional de wallet XCOP y timeline, manejo uniforme de errores y base mínima de no regresión.

6. Riesgos de no ejecutar esta hoja de ruta

Postergar estos frentes mantendría la distancia entre apariencia de madurez y comportamiento real del frontend.

- Seguir ampliando alcance sobre una base con navegación ambigua y validaciones debilitadas incrementará la probabilidad de defectos transversales difíciles de detectar.
- La UI continuará comunicando una completitud funcional mayor a la realmente soportada, especialmente en flujos cripto y de seguimiento transaccional.
- La dependencia de piezas grandes, helper transversal y ausencia de pruebas hará que cada cambio nuevo tenga mayor costo de revisión, mayor riesgo de ruptura y menor trazabilidad.
- Cualquier reevaluación seria de salida a producción seguirá bloqueada mientras el frontend no pueda demostrar seguridad básica del cliente, representación honesta del flujo crítico y controles mínimos de calidad.

Conclusión ejecutiva: esta hoja de ruta no busca embellecer el frontend, sino convertirlo en una base de trabajo defendible. La prioridad es corregir primero lo que compromete seguridad, confianza operativa y riesgo de regresión. Solo después tiene sentido retomar expansión funcional o refinamiento visual.

Creado por Jose Manuel Guerrero Duarte, Ing en informática